

SQL Injection UNION Attack: Retrieving Multiple Values in a Single Column

1. Vulnerability Overview

- **Vulnerability Type:** SQL Injection — UNION Attack / String Concatenation
- **Security Impact:** Unauthorized Data Extraction
- **Primary Objective:** Retrieve multiple data fields, such as usernames and passwords, when the application exposes only a single suitable output column.

2. Vulnerability Description

In some UNION-based SQL injection scenarios, the application's response may expose only a single column capable of displaying textual data.

When multiple sensitive fields need to be retrieved, an attacker may attempt to combine multiple database values into a single output string using database-specific string concatenation functionality.

Depending on the database management system, concatenation may be performed using operators such as `||` or functions such as `CONCAT()`.

3. Attack Vector & Exploitation Methodology

The assessment first identifies the number of columns returned by the vulnerable query and determines which column is reflected in the application's response.

The tester then evaluates whether multiple database fields can be combined into a single textual value and returned through the available output column.

This technique can potentially allow multiple sensitive values to be exposed through a response field that would normally display only one database value.

4. Security Impact

Successful exploitation may result in:

- Unauthorized extraction of multiple database fields.
- Exposure of usernames and password-related information.
- Disclosure of sensitive application records.
- Increased effectiveness of subsequent database compromise attempts.

5. Root Cause

The underlying root cause is SQL query construction that allows attacker-controlled input to alter query structure.

The vulnerability is further enabled when application output directly reflects database results without appropriate access controls and secure query handling.

6. Remediation

- Use **parameterized queries / prepared statements** for all database operations.
- Prevent user-controlled input from modifying SQL query structure.
- Apply the **Principle of Least Privilege** to database accounts.
- Use secure database-access abstractions or **Object-Relational Mapping (ORM)** frameworks where appropriate.
- Avoid exposing unnecessary database content through application responses.

7. Security Recommendation

Prepared statements should remain the primary defense against SQL injection. ORM frameworks can provide an additional abstraction layer, but they must still be used correctly and should not be treated as a substitute for secure authorization and input-handling practices.

Author: Moe Htet Ar Kar (Phoe Cho)